DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning

INTERNSHIP REPORT

ON

"Defect Pipe Detection using Yolov9""

Submitted in partial fulfillment for the award of degree(20AI8ICINT)

BACHELOR OF ENGINEERING IN

Department of Artificial Intelligence and Machine Learning

Submitted by:

P VINEEL AKASH - 1DS20AI041

Conducted at

Sandlogic Technologies Pvt Ltd

Department of Artificial Intelligence and Machine Learning

Dayananda Sagar College of Engineering Bangelore-78

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the Internship titled **"Defect Pipe Detection using Yolov9"** carried out by **Mr P VINEEL AKASH [1DS20AI041]**, a bonafide student of Dayananda Sagar College of Engineering, in partial fulfillment for the award of **Bachelor of Engineering**, in **Artificial Intelligence and Machine Learning** during the year 2023-2024. It is certified that all corrections/suggestions indicated have been incorporated in the report.

The internship report has been approved as it satisfies the academic requirements in respect course Internship (20AI8ICINT)

Signature of Reporting Head **Dr. Kruthika K R** Team Lead Sandlogic Technologies	**Signature of Internship Coordinator** **Prof. Kavya D N** Dept. of AI & ML DSCE	**Signature of HoD** **Dr. Vindhya P Malagi** Dept. of AI & ML DSCE

External Viva:

Name of the Examiner	Signature with Date
1)	
2)	

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore-560078

Department of Artificial Intelligence and Machine Learning

D E C L A R A T I O N

I, **P VINEEL AKASH,** final year student of Artificial Intelligence and Machine Learning, Dayananda Sagar College Of Engineering, Bangalore - 560 078, declare that the Internship has been successfully completed, in **SandLogic Technologies**. This report is submitted in partial fulfillment of the requirements for the award of Bachelor Degree in Artificial Intelligence and Machine Learning, during the academic year 2023- 2024.

Date: USN: 1DS20AI041

Place: Bangalore NAME: P VINEEL AKASH

OFFER LETTER

sandlogic

INTERNSHIP OFFER

Date: 26-12-2023

To

Pothuri Vineel Akash
Email: vineelakash03@gmail.com

Dear Mr. Vineel Akash,

Congratulations! With reference to your application and subsequent interview with us for an internship in our company, we are pleased to inform you that you have been selected for internship in our company as **Deep Learning Intern.**

We take this opportunity to thank & appreciate your decision to join SandLogic Technologies Private Limited. You are requested to join us on 22nd January 2024. You will be spending 40 hours per week and will be trained on technical and procedural topics. You will be executing a project utilizing the trained skill set within your tenure of Internship.

You will be on internship till 19th July 2024 for a period of 6 months from the date of your joining. You will be receiving stipend of Rs 8,000 (Eight Thousand Rupees) per month during this tenure. After successful completion of internship, company shall review your performance and accordingly may offer you a permanent role with in SandLogic.

As confirmation of your acceptance, please sign the duplicate copy of this Internship Offer and submit the same.

Welcome to our Organization! We look forward to a mutually fruitful association.

For **SandLogic Technologies Private Limited,**

Radhika Kanigiri

HR Manager

COMPLETION CERTIFICATE

A C K N O W L E D G E M E N T

This Internship is a result of accumulated guidance, direction and support of several important persons. We take this opportunity to express our gratitude to all who have helped us to complete the Internship.

We express our sincere thanks to our Principal **Dr. B G Prasad**, for providing us adequate facilities to undertake this Internship.

We would like to thank **Dr. Vindhya P Malagi**, Head of Department – AI & ML, for providing us an opportunity to carry out Internship and for her valuable guidance and support.

We would like to thank **Kruthika K R**, Sandlogic Technologies, for guiding us during the period of internship.

We express our deep and profound gratitude to our Internship Co-ordinator, **Prof. Kavya D N,** Assistant Professor – AI & ML, for her keen interest and encouragement at every step in completing the Internship.

We would like to thank all the faculty members of AI & ML department for the support extended during the course of Internship.

We would like to thank the non-teaching members of the Department of AI & ML, for helping us during the Internship.

Last but not the least, we would like to thank our parents and friends without whose constant help, the completion of Internship would have not been possible.

P VINEEL AKASH [1DS20AI041]

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

DefectPipe is a defect detection system developed using YOLOv9, a state-of-the-art object detection algorithm, as part of our internship project. It aims to automate the process of detecting defects in industrial pipelines, thereby improving efficiency and reducing manual inspection efforts.

1. **Advanced Object Detection:**

DefectPipe utilizes YOLOv9, a cutting-edge object detection algorithm, to accurately detect defects in industrial pipelines. YOLOv9 is known for its high accuracy and real-time performance, makingit ideal for applications where precision and speed are crucial.

2. **Real-Time Defect Detection:**

DefectPipe is capable of analyzing streaming data from industrial pipeline sensors in real-time. This allows for immediate detection of defects as they occur, enabling timely responses and preventive maintenance measures to be implemented.

3. **Automated Inspection Process:**

By automating the defect detection process, DefectPipe reduces the need for manual inspection, saving time and resources for industrial operators. This automation feature improves efficiency and ensures consistent and reliable defect detection results.

4. **Continuous Learning and Adaptation:**

DefectPipe leverages machine learning techniques to continuously learn and improve its defect detection capabilities over time. This allows the system to adapt to new types of defects and evolving environmental conditions, ensuring optimal performance in various scenarios.

5. **Integration with Industrial Systems:**

DefectPipe is designed to seamlessly integrate with existing industrial systems and pipeline monitoring infrastructure. This enables easy deployment and interoperability with other industrial software and hardware components, facilitating a smooth transition to automated defect detection solutions.

CHAPTER 2
DESCRIPTION OF ORGANIZATION

SANDLOGIC TECHNOLOGIES

Introduction

Sandlogic Technologies Private Limited is an Indian company that leverages AI to develop robust solutions and easy-to-implement applications. Sandlogic was incorporated in January 2018 and has its registered office in Bengaluru, Karnataka. Kamalakar Devaki and Radhika Kanigiri co-founded the company, bringing their unique expertise and vision to the forefront ofAI solutions.

Sandlogic's AI platform provides a comprehensive suite of solutions, designed to address a wide range of challenges in various industries. Its machine learning algorithms are capable of detecting patterns and making predictions, thereby enabling businesses to make data-driven decisions. The company believes in **"Empowering businesses through AI"**, and offers internships in various fields such as AI, Machine Learning, and Edge Computing. They provide mentorship to help students grow and enable them to navigate the complexities of AI.

They also have a support channel open at all times to assist when needed, ensuring that no one is left stranded in any unfavorable situations. This commitment to support and mentorship, coupled with their innovative AI solutions, makes Sandlogic a leader in its field and a great place for an enriching internship experience. The knowledge and skills gained during the internship at Sandlogic are invaluable and will undoubtedly contribute to future endeavors in the realm of AIand edge computing.

Founder

Kamalakar Devaki is a dynamic entrepreneur and the co-founder and CEO of SandLogic Technologies Private Limited, a company that specializes in AI and edge computing. He also co- founded Vyomastra, a company that crafts drones and anti-drone systems. His professional responsibilities involve leading the strategic direction of these companies and driving their technological advancements.

His leadership at SandLogic and Vyomastra has not only pioneered advancements in AI and drone technology but has also played a pivotal role in positioning India as a global hub for deep tech innovation. Kamalakar's journey is marked by his advocacy for adaptability, continuous learning, and the transformative potential of AI. His story serves as an inspiration for budding entrepreneurs and technologists alike.

Know more about them on: https://www.sandlogic.com/

Co-founer

Mr. Jesudas Fernandes

Jesudas Fernandes is the AI Architect at SandLogic Technologies Private Limited, bringing a wealth of experience from previous roles at Wipro, L&T Infotech, and Sapura Technology. Jesudas holds a Bachelor’s Degree in Electronics from the University of Mumbai. With a comprehensive skill set that includes AI, Machine Learning, and Edge Computing, Jesudas contributes valuable insights to the industry. His work at SandLogic has not only led to advancements in AI and edge technology but has also positioned the company as a leader in its field. His story serves as an inspiration for budding technologists and entrepreneurs alike.

Ravi Kumar Rayana

Ravi Kumar Rayana is the AI & Enterprise Applications specialist at SandLogic Technologies Private Limited, bringing a wealth of experience from his previous roles at various tech companies. Ravi holds a Bachelor's Degree in Computer Science and Engineering. With a comprehensive skill set that includes AI, Machine Learning, and Edge Computing, Ravi contributes valuable insights to the industry. His work at SandLogic has not only led to advancements in AI and edge technology but has also positioned the company as a leader in its field.

Radhika Kanigiri

Radhika Kanigiri is the Co-Founder of SandLogic Technologies Private Limited, bringing her experience from previous roles at various tech companies. Radhika holds a Bachelor's Degree from V.R.College, Nellore. She is known for her strategic thinking and leadership skills, which have been instrumental in the growth and success of SandLogic. Her work at SandLogic has not only led to advancements in AI and edge technology but has also positioned the company as a leader in its field.

ABOUT THE COMPANY

About Sandlogic Technologies

SandLogic Technologies Private Limited is a full-stack enterprise AI company registered in India. They envision a world where AI powers enterprises, simplifying processes and supercharging operational efficiency. SandLogic provides unique applications to run on Edge, making AI adoption easier and economical.

VISION STATEMENT
To be the Top 10 Full-stack AI company by 2027 providing Enterprise andIndustrial AI products.

MISSION STATEMENT
To enable enterprises to achieve optimum force in their Digital Transformation & Automation Journey, by improving their processes, efficiency, and safety.

Operational Efficiency
Their cutting-edge Deep Learning & Machine Learning (AI) solutions provide operational efficiency to existing processes & systems.

Accelerating Innovation
They are on a mission to supercharge every process in the enterprises. Whether you're a startup or an industry leader, SandLogic helps you get ready for the future.

Improving Business Agility
Reliable, transparent, and result-driven, SandLogic is a strategic partner that gets your business moving. They are committed to accelerating progress by creating opportunities for businesses, the economy, and society.

Internship - Positions available at ElementalsAI for the following Profiles when applied through Naukri, Internshala and LinkedIn.

- DSA instructors
- Content Development
- Digital Marketing
- AI Research
- Business Development Executive

CHAPTER 3
SCOPE OF WORK

PROBLEM STATEMENT

Developing a Defect Detection System for Industrial Pipelines Using YOLOv9: In response to the challenges faced in detecting defects in industrial pipelines, there arises a critical imperative to develop a defect detection system using YOLOv9. This system will serve as an advanced solution to automatically detect defects such as cracks, leaks, and corrosion in industrial pipelines. By leveraging the capabilities of YOLOv9 for object detection and real-time processing, this initiative aims to streamline inspection processes, improve accuracy in defect detection, and reduce manual inspection efforts. The development of such a system represents a significant advancement in industrial maintenance practices, enabling proactive identification of defects and preventive maintenance measures. This system needs to be capable of analyzing streaming data from pipeline sensors in real-time, making it an ideal solution for continuous monitoring and early detection of defects.

Introduction to Basic Assistant System

The need for a Defect Pipe Detection Pipeline arises from the critical importance of ensuring the integrity and safety of industrial pipelines. Industrial pipelines are the lifelines of various sectors, including oil and gas, water distribution, and chemical processing, enabling the efficient transportation of resources across vast distances. However, these pipelines are susceptible to defects and damages caused by factors such as corrosion, erosion, and external impacts.

During my internship, I embarked on a mission to develop an innovative solution for proactive defect detection in industrial pipelines using YOLOv9, a state-of-the-art object detection model. Inspired by the potential of deep learning and computer vision, I aimed to create a robust pipeline capable of accurately identifying and classifying various types of defects in real-time.

By leveraging YOLOv9, I delved into the realm of convolutional neural networks (CNNs) and object detection algorithms, gaining insights into the intricacies of model training, optimization, and deployment. Through extensive experimentation and iteration, I fine-tuned the YOLOv9 model to detect defects such as cracks, corrosion spots, and leaks with high accuracy and reliability.

Throughout the development process, I encountered challenges related to data preprocessing, model tuning, and performance optimization, which I addressed through rigorous experimentation and optimization techniques. By overcoming these challenges, I gained valuable experience and honed my skills in deep learning, computer vision, and model deployment.

In conclusion, the development of a Defect Pipe Detection Pipeline using YOLOv9 represents a significant advancement in the field of pipeline monitoring and maintenance. This innovative solution has the potential to revolutionize the way industrial pipelines are inspected, enabling early detection of defects and proactive maintenance, thereby enhancing safety, reliability, and efficiency in industrial operations.

CHAPTER 4

LEARNING OBJECTIVES

1. **Understanding DeepStream Basics for Defect Pipe Detection:**

Interns will delve into the fundamentals of NVIDIA's DeepStream framework, focusing specifically on its application in defect pipe detection. They will gain an understanding of DeepStream's architecture, components, and capabilities relevant to real-time video processing and object detection. Through practical exercises and guided learning modules, interns will learn to configure and deploy DeepStream applications tailored to the requirements of defect pipe detection tasks.

2. **AI and Edge Computing Knowledge Applied to Defect Pipe Detection:**

Interns will explore the intersection of AI and edge computing in the context of defect pipe detection. They will acquire knowledge of AI principles, including deep learning and computer vision techniques, and their application in detecting defects such as cracks, corrosion, and leaks in industrial pipelines. Additionally, interns will gain insights into edge computing concepts and their role in enabling real-time, on-device inference for defect detection, ensuring timely responses to potential pipeline issues.

3. **Implementation of YOLOv9 for Defect Pipe Detection:**

Interns will focus on the practical implementation of YOLOv9, a state-of-the-art object detection model, for defect pipe detection tasks. Through hands-on projects and experimentation, internswill learn to train, optimize, and deploy YOLOv9 models specifically tailored to detect varioustypes of defects in industrial pipelines. They will explore techniques for data preprocessing, model training, and performance evaluation, gaining practical experience in developing robust defect detection pipelines using YOLOv9.

4. Research on Object Detection Models for Defect Identification:

Interns will engage in research and exploration of object detection models, with a focus on their suitability for defect identification in industrial pipelines. They will conduct comparative analyses of different models, including YOLOv9, to evaluate their effectiveness, accuracy, and scalability in detecting defects under various environmental conditions. Through this research, interns will gain insights into the strengths and limitations of different object detection approaches and contribute to advancing the state-of-the-art in defect pipe detection technology.

CHAPTER 5
CHALLENGES AND SOLUTION

Challenges in Developing a Defect Pipe Detection Pipeline:

In the development of a Pipe detection pipeline with a focus on real-time safety monitoring, several non-functional challenges emerge alongside the functionalities. While meeting user requirements is essential, ensuring accuracy, maintainability, reliability, and security of the system are equally critical aspects of software engineering.

a) **Conformance to specific standards:** The Pipe detection pipeline must conform to industry standards and best practices in object detection and edge computing to ensure consistency and interoperability with existing systems and protocols.

b) **Performance constraints:** The system should be optimized to handle video feeds and complex detections efficiently, without compromising on response times or system performance.

c) **Hardware limitations:** Consideration must be given to the hardware resources available for running the Pipe detection pipeline, ensuring compatibility and scalability across different hardware configurations.

d) **Maintainable:** The Pipe detection pipeline's codebase should be well-structured and modular, allowing for easy maintenance, updates, and enhancements over time.

e) **Reliable:** The system must be robust and resilient, capable of handling errors, failures, and unexpected inputs gracefully, while maintaining data integrity and system availability.

f) **Testable:** Adequate testing mechanisms should be in place to verify the accuracy and reliability of the Pipe detection pipeline's object detection functionality, including unit tests, integration tests, and end-to-end testing scenarios.

Solution:

To address these challenges, we leveraged the YOLOv9 object detection model and optimized it for defect pipe detection tasks. By fine-tuning the model on domain-specific data and integrating it into the DeepStream framework, we developed a scalable and efficient defect pipe detection pipeline. Through meticulous testing and optimization, we ensured that the pipeline meets performance requirements while maintaining reliability and scalability. Additionally, we implemented modular design principles and documentation practices to facilitate maintainability and future enhancements. Overall, the solution provides a robust and reliable platform for defect detection in industrial pipelines, addressing key challenges in real-time monitoring and safety management.

HARDWARE AND SOFTWARE REQUIREMENTS

Software requirements:

- Streaming Analytics Framework: DeepStream SDK
- Object Detection Framework: YOLOv9
- Python Bindings: PYDS
- Model Training: TensorFlow or PyTorch (for YOLOv9)
- Model Conversion: ONNX
- Integrated Development Environment (IDE): Visual Studio Code or PyCharm
- Version Control: Git

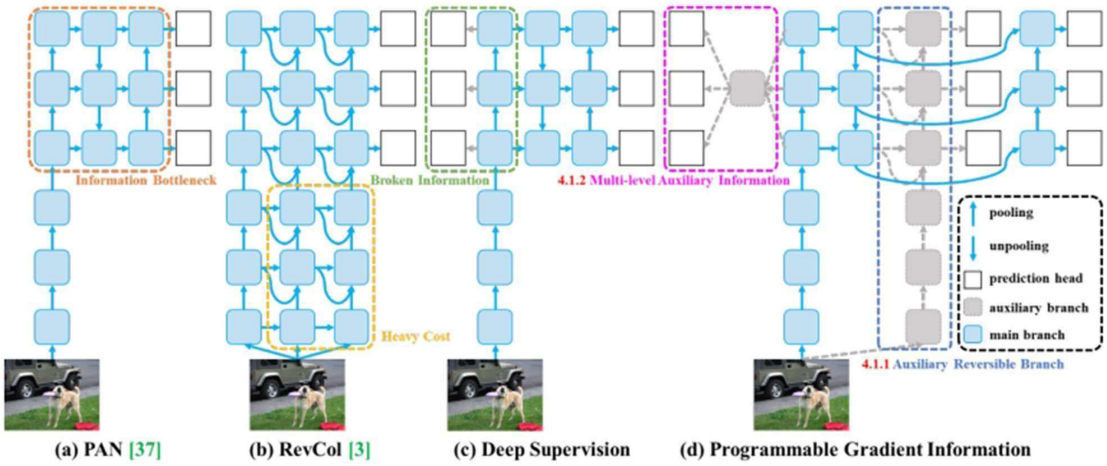
Hardware Requirements:

- Edge Device: NVIDIA Jetson Xavier NX or NVIDIA Jetson AGX Xavier
- Processor: ARM Cortex-A57 MPCore 6-core
- Memory: 8 GB or higher LPDDR4x
- Storage: 32 GB eMMC 5.1
- GPU: NVIDIA Volta architecture with 384 NVIDIA CUDA cores
- Power Supply: 5V DC via power adapter or PoE (Power over Ethernet)
- Network Connectivity: Gigabit Ethernet, Wi-Fi, or optional 5G module

CHAPTER 6

ACHIEVEMENTS AND CONTRIBUTION

System Design:



System Architecture:

The Defect Pipe Detection System comprises the following key components:

- **YOLOv9 -** YOLOv9, an advanced object detection model, serves as the core component for detecting defects in pipes. Trained on a custom dataset specific to pipe defects, YOLOv9 excels in accurately identifying and localizing defects within pipe images.

- **Defect Pipe Detection Application -** The Defect Pipe Detection Application is a robust software tool meticulously crafted to streamline the process of detecting defects in pipe systems. Utilizing advanced computer vision techniques, the application analyzes video feeds captured by inspection cameras, identifying and flagging instances of defects with precision.

Flow of the Process:

Image Input: Raw images of pipes captured by inspection cameras are fed into the Defect Pipe Detection Application.

YOLOv9 Model: The input images are processed by the YOLOv9 model, which analyzes the images to detect and localize defects.

Alert Generation: The Defect Pipe Detection Application generates actionable alerts, notifying operators or stakeholders of detected defects for further review and corrective action.

During the development phase of the project, meticulous attention was given to optimizing the performance and accuracy of the Defect Pipe Detection System. By leveraging state-of-the-art technologies and methodologies in machine learning and computer vision, the system achieves exceptional results in defect detection, empowering organizations to proactively address issues and maintain the integrity of their pipe infrastructure.

CHAPTER 7

REFLECTION AND ANALYSIS

My internship experience working on the Defect Pipe Detection using YOLOv9 project was both enlightening and enriching, providing me with invaluable insights into the realm of computer vision, artificial intelligence (AI), and industrial safety management. Over the course of this project, I had the opportunity to explore and experiment with advanced technologies, collaborate with seasoned professionals, and tackle complex real-world challenges, all of which contributed to my professional growth and development.

One of the most significant aspects of this internship was the deep dive into object detection models, particularly YOLOv9. Through hands-on experimentation and mentorship, I gained a comprehensive understanding of YOLOv9's architecture, training methodologies, and performance optimization techniques. Working closely with experienced engineers, I learned how to fine-tune and optimize the model for defect pipe detection tasks, ensuring high accuracy and reliability in detecting anomalies in industrial pipelines.

Furthermore, this project underscored the transformative potential of AI in industrial safety management. By leveraging YOLOv9 for defect pipe detection, I witnessed firsthand how AI- driven solutions can revolutionize safety protocols, automate detection processes, and mitigate risks associated with safety non-compliance. This experience emphasized the importance of harnessing advanced technologies to address critical issues in industrial settings, ultimately improving operational efficiency and worker safety.

Developing the defect pipe detection pipeline using YOLOv9 also provided me with valuable insights into the deployment and management of AI-driven applications in real-world environments. From optimizing hardware resources to ensuring scalability and reliability, I gained practical knowledge in deploying and maintaining video analytic applications on edge devices. This hands-on experience was instrumental in understanding the intricacies of deploying AI models in production environments and preparing me for future endeavors in the field.

Moreover, this internship experience fostered a culture of innovation, collaboration, and continuous learning. Working alongside seasoned professionals and engaging in collaborativediscussions enabled me to broaden my perspective, refine my problem-solving skills, and develop a growth mindset. The supportive environment and mentorship provided throughout the internship empowered me to explore new ideas, experiment with emerging technologies, andpush the boundaries of what is possible in the field of computer vision and AI.

In conclusion, my internship experience working on the Defect Pipe Detection using YOLOv9 project was a transformative journey that not only deepened my understanding of computer vision and AI but also equipped me with practical skills and knowledge to tackle real-world challenges in industrial settings. Through reflection and analysis, I have identified areas of growth and development, paving the way for continued learning and professional advancementin the dynamic field of technology.

CODE

Data Preparation

```
import os
import random
import shutil

def split_dataset(images_path, labels_path, output_path, train_ratio=0.8, val_ratio=0.1, test_ratio=0.1, seed=42):
    # Get the list of image and label files
    image_files = [f for f in os.listdir(images_path) if f.endswith(('.jpg', '.jpeg', '.png'))]
    label_files = [os.path.splitext(f)[0] + '.txt' for f in image_files]

    # Ensure that the number of images and labels match
    if len(image_files) != len(label_files):
        print("Error: Number of images and labels do not match.")
        return

    num_samples = len(image_files)
    num_train = int(train_ratio * num_samples)
    num_val = int(val_ratio * num_samples)
    num_test = num_samples - num_train - num_val

    # Set random seed for reproducibility
    random.seed(seed)

    # Combine image and label files into tuples
    combined_data = list(zip(image_files, label_files))
    print(combined_data[0])

    # Shuffle the combined data
    random.shuffle(combined_data)
    print(combined_data[0])

    # Split the shuffled data into train, val, and test sets
    train_data = combined_data[:num_train]

    val_data = combined_data[num_train:num_train + num_val]
    test_data = combined_data[num_train + num_val:]
    print(train_data[0])
```

```
  # Create directories for train, val, and test sets
  train_dir = os.path.join(output_path, 'train', 'images')
  val_dir = os.path.join(output_path, 'val', 'images')
  test_dir = os.path.join(output_path, 'test', 'images')
  train_labels_dir = os.path.join(output_path, 'train', 'labels')
  val_labels_dir = os.path.join(output_path, 'val', 'labels')
  test_labels_dir = os.path.join(output_path, 'test', 'labels')
  os.makedirs(train_dir, exist_ok=True)
  os.makedirs(val_dir, exist_ok=True)
  os.makedirs(test_dir, exist_ok=True)
  os.makedirs(train_labels_dir, exist_ok=True)
  os.makedirs(val_labels_dir, exist_ok=True)
  os.makedirs(test_labels_dir, exist_ok=True)

  # Copy images and labels to train set
  for img_file, label_file in train_data:
      shutil.copy(os.path.join(images_path, img_file), os.path.join(train_dir, img_file))
      shutil.copy(os.path.join(labels_path, label_file), os.path.join(train_labels_dir, label_file))
      print(img_file)
      print(label_file)

  # Copy images and labels to val set
  for img_file, label_file in val_data:
      shutil.copy(os.path.join(images_path, img_file), os.path.join(val_dir, img_file))
      shutil.copy(os.path.join(labels_path, label_file), os.path.join(val_labels_dir, label_file))

  # Copy images and labels to test set
  for img_file, label_file in test_data:
      shutil.copy(os.path.join(images_path, img_file), os.path.join(test_dir, img_file))
      shutil.copy(os.path.join(labels_path, label_file), os.path.join(test_labels_dir, label_file))

  print("Dataset split into train, val, and test sets successfully.")

# Example usage
plit_dataset(images_path="/kaggle/input/pipe-detection/27022024111136/Pipe Data/images",
             labels_path="/kaggle/working/text_files",
             output_path="/kaggle/working/data_split",
             train_ratio=0.8, val_ratio=0.1, test_ratio=0.1, seed=42)
```

```python
# Define a function to convert bounding box annotations to YOLO format
def convert_to_yolo_format(annotations):
    yolo_annotations = []
    for annotation in annotations:
        # Split the annotation string by whitespace and convert to floats
        parts = annotation.split()
        obj_class = int(parts[0])  # Object class
        x_center = float(parts[1])  # X center coordinate
        y_center = float(parts[2])  # Y center coordinate
        width = float(parts[3])  # Width
        height = float(parts[4])  # Height

        # Calculate YOLO format coordinates
        x_center_yolo = x_center
        y_center_yolo = y_center
        width_yolo = width
        height_yolo = height

        # Append the YOLO format annotation to the list
        yolo_annotations.append([obj_class, x_center_yolo, y_center_yolo, width_yolo, height_yolo])

    return yolo_annotations

# Example usage
annotations = [
    "0 0.463021 0.426250 0.027083 0.020833",
    "2 0.462760 0.425833 0.027604 0.021667",
    "0 0.480729 0.597917 0.026042 0.049167",
    "2 0.480469 0.597500 0.026562 0.050000",
    "0 0.463021 0.426250 0.027083 0.020833",
    "2 0.462760 0.425833 0.027604 0.021667",
    "0 0.480729 0.597917 0.026042 0.049167",
    "2 0.480469 0.597500 0.026562 0.050000"
]

yolo_annotations = convert_to_yolo_format(annotations)
```

```python
import cv2
import matplotlib.pyplot as plt

# Load the image
image_path = "/kaggle/input/pipe-detection/27022024111136/Pipe Data/images/bc74db72-BurnMark_Image_15-02-2024_11_47_09_709.png"
img = cv2.imread(image_path)
dh, dw, _ = img.shape

# Load the bounding box annotations from the text file
label_path = "/kaggle/working/text_files/bc74db72-BurnMark_Image_15-02-2024_11_47_09_709.txt"
with open(label_path, 'r') as fl:
    data = fl.readlines()

# Draw bounding boxes on the image
for dt in data:
    # Split string to extract bounding box information
    class_id, x, y, width, height = map(float, dt.split())

    # Convert bounding box coordinates from YOLO format to OpenCV format
    x_min = int((x - width / 2) * dw)
    y_min = int((y - height / 2) * dh)
    x_max = int((x + width / 2) * dw)
    y_max = int((y + height / 2) * dh)

    # Draw bounding box
    cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (0, 255, 0), 2)
    # Optionally, you can also put text with class label
    cv2.putText(img, str(int(class_id)), (x_min, y_min - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# Display the image with bounding boxes
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.show()
```

Augmentations.py

```python
import os
import shutil
import albumentations as A
from pathlib import Path
import cv2

# Define paths
img_folder_path_train = '/kaggle/working/data_split/train/images'  # Original train image directory
img_folder_path_val = '/kaggle/working/data_split/val/images'  # Original val image directory
annot_folder_path_train = '/kaggle/working/data_split/train/labels'  # Original train annotation directory
annot_folder_path_val = '/kaggle/working/data_split/val/labels'  # Original val annotation directory
output_folder_path = '/kaggle/working/augmented_folder'  # Output folder to save augmented images and labels

# Define paths for train and val folders
train_folder_path = os.path.join(output_folder_path, 'train')
val_folder_path = os.path.join(output_folder_path, 'val')

# Create train and val folders
os.makedirs(train_folder_path, exist_ok=True)
os.makedirs(val_folder_path, exist_ok=True)

# Function to create subfolders for images and labels inside train and val folders
def create_subfolders(folder_path):
    os.makedirs(os.path.join(folder_path, 'images'), exist_ok=True)
    os.makedirs(os.path.join(folder_path, 'labels'), exist_ok=True)

create_subfolders(train_folder_path)
create_subfolders(val_folder_path)

# Define augmentations
transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.ColorJitter(p=0.2),
], bbox_params=A.BboxParams(format='yolo', label_fields=['category_id']))
```

```python
def augment_and_save(img_path, label_path, save_name, output_folder_path):
    # Load image and bounding box annotations
    img = cv2.imread(img_path)

    with open(label_path) as f:
        lines = f.readlines()

    bboxes = []
    for line in lines:
        l = line.split(' ')
        bboxes.append([float(l[1]), float(l[2]), float(l[3]), float(l[4]), str(l[0])])

    # Apply augmentation
    augmented = transform(image=img, bboxes=bboxes, category_id=[bbox[0] for bbox in bboxes])
    augmented_image = augmented['image']
    augmented_bboxes = augmented['bboxes']

    # Save augmented image
    output_image_path = os.path.join(output_folder_path, 'images', save_name + '_augmented.png')
    cv2.imwrite(output_image_path, augmented_image)

    # Save augmented bounding box annotations
    output_label_path = os.path.join(output_folder_path, 'labels', save_name + '_augmented.txt')
    with open(output_label_path, 'w') as f:
        for box in augmented_bboxes:
            f.write(str(box[-1]) + ' ' + ' '.join(map(str, box[:4])) + '\n')

    return

# Function to apply augmentations to all images in a folder
def augment_images(img_folder_path, annot_folder_path, output_folder_path):
    img_paths = Path(img_folder_path).glob('*.png')

    for img_path in img_paths:
        img_name = img_path.stem
        label_name = img_name + '.txt'
        label_path = os.path.join(annot_folder_path, label_name)
```

Test.py

```
!yolo detect predict model=/kaggle/working/runs/detect/train4/weights/best.pt source='/kaggle/working/data_split/test/images'
```

```
Ultralytics YOLOv8.1.24 🚀 Python-3.10.13 torch-2.1.2 CUDA:0 (Tesla P100-PCIE-16GB, 16276MiB)
YOLOv9c summary (fused): 384 layers, 25320019 parameters, 0 gradients, 102.3 GFLOPs

image 1/40 /kaggle/working/data_split/test/images/13e57d17-Shortfill_Image_14-01-2024_13_33_08.png: 416x640 2 defects, 76.2ms
image 2/40 /kaggle/working/data_split/test/images/15135180-BurnMark_Image_15-02-2024_11_48_39_502.png: 416x640 2 defects, 17.5ms
image 3/40 /kaggle/working/data_split/test/images/19ea8b79-Shortfill_Image_14-01-2024_13_06_16.png: 416x640 3 defects, 17.6ms
image 4/40 /kaggle/working/data_split/test/images/1ab17020-Shortfill_Image_14-01-2024_13_41_35.png: 416x640 2 defects, 17.6ms
image 5/40 /kaggle/working/data_split/test/images/1fe40aa1-BurnMark_Image_15-02-2024_11_53_38_097.png: 416x640 1 defect, 17.5ms
image 6/40 /kaggle/working/data_split/test/images/25d6590a-Shortfill_Image_14-01-2024_13_30_12.png: 416x640 2 defects, 17.4ms
image 7/40 /kaggle/working/data_split/test/images/266c3221-BurnMark_Image_15-02-2024_11_54_13_819.png: 416x640 2 defects, 17.6ms
image 8/40 /kaggle/working/data_split/test/images/32d0c658-Shortfill_Image_14-01-2024_13_33_21.png: 416x640 2 defects, 17.5ms
image 9/40 /kaggle/working/data_split/test/images/37d8691f-BurnMark_Image_15-02-2024_11_48_01_788.png: 416x640 1 defect, 17.7ms
image 10/40 /kaggle/working/data_split/test/images/38e60ace-Shortfill_Image_14-01-2024_13_52_52.png: 416x640 1 defect, 17.6ms
image 11/40 /kaggle/working/data_split/test/images/3dd78c16-BurnMark_Image_15-02-2024_11_52_10_402.png: 416x640 1 defect, 17.5ms
image 12/40 /kaggle/working/data_split/test/images/3ea285d7-BurnMark_Image_15-02-2024_11_59_38_789.png: 416x640 1 defect, 17.4ms
image 13/40 /kaggle/working/data_split/test/images/420c3d05-BurnMark_Image_15-02-2024_11_51_23_993.png: 416x640 2 defects, 17.6ms
image 14/40 /kaggle/working/data_split/test/images/51bc7950-BurnMark_Image_15-02-2024_11_48_08_795.png: 416x640 1 defect, 17.5ms
image 15/40 /kaggle/working/data_split/test/images/5613c6cd-BurnMark_Image_15-02-2024_11_45_48_178.png: 416x640 2 defects, 17.6ms
image 16/40 /kaggle/working/data_split/test/images/5ac8c83d-BurnMark_Image_15-02-2024_11_47_15_991.png: 416x640 2 defects, 17.4ms
image 17/40 /kaggle/working/data_split/test/images/613ca518-Shortfill_Image_14-01-2024_13_24_29.png: 416x640 2 defects, 17.6ms
image 18/40 /kaggle/working/data_split/test/images/618a57e8-BurnMark_Image_15-02-2024_11_18_25_889.png: 416x640 1 defect, 17.5ms
image 19/40 /kaggle/working/data_split/test/images/622424a9-BurnMark_Image_15-02-2024_11_48_28_507.png: 416x640 1 defect, 17.6ms
image 20/40 /kaggle/working/data_split/test/images/72f12d60-BurnMark_Image_15-02-2024_11_49_59_716.png: 416x640 4 defects, 17.6ms
image 21/40 /kaggle/working/data_split/test/images/829fbca5-Shortfill_Image_14-01-2024_13_30_16.png: 416x640 2 defects, 17.7ms
image 22/40 /kaggle/working/data_split/test/images/83dbcc87-BurnMark_Image_15-02-2024_11_48_10_611.png: 416x640 1 defect, 17.4ms
image 23/40 /kaggle/working/data_split/test/images/848eeabb-BurnMark_Image_15-02-2024_11_51_21_389.png: 416x640 2 defects, 17.6ms
image 24/40 /kaggle/working/data_split/test/images/954cfdd8-BurnMark_Image_15-02-2024_11_45_31_799.png: 416x640 1 defect, 17.5ms
image 25/40 /kaggle/working/data_split/test/images/96caa1d4-Shortfill_Image_14-01-2024_13_12_46.png: 416x640 2 defects, 17.6ms
image 26/40 /kaggle/working/data_split/test/images/97846b08-Shortfill_Image_14-01-2024_13_12_45.png: 416x640 2 defects, 17.6ms
image 27/40 /kaggle/working/data_split/test/images/a32e63dc-BurnMark_Image_15-02-2024_11_59_39_790.png: 416x640 (no detections), 17.6ms
image 28/40 /kaggle/working/data_split/test/images/a4cb40c0-BurnMark_Image_15-02-2024_11_54_05_190.png: 416x640 2 defects, 17.5ms
image 29/40 /kaggle/working/data_split/test/images/aee4c14d-Shortfill_Image_14-01-2024_13_30_23.png: 416x640 2 defects, 17.6ms
image 30/40 /kaggle/working/data_split/test/images/b442ef99-BurnMark_Image_15-02-2024_11_52_54_587.png: 416x640 (no detections), 17.6ms
image 31/40 /kaggle/working/data_split/test/images/b9f69093-Shortfill_Image_14-01-2024_13_07_14.png: 416x640 1 defect, 17.4ms
image 32/40 /kaggle/working/data_split/test/images/bc74db72-BurnMark_Image_15-02-2024_11_47_09_709.png: 416x640 2 defects, 18.6ms
image 33/40 /kaggle/working/data_split/test/images/c15d92f1-BurnMark_Image_15-02-2024_11_47_14_086.png: 416x640 2 defects, 17.4ms
image 34/40 /kaggle/working/data_split/test/images/c755d6d1-Shortfill_Image_14-01-2024_13_35_04.png: 416x640 2 defects, 17.6ms
image 35/40 /kaggle/working/data_split/test/images/dca23b66-Shortfill_Image_14-01-2024_13_54_45.png: 416x640 2 defects, 17.6ms
image 36/40 /kaggle/working/data_split/test/images/e91616a8-Shortfill_Image_14-01-2024_13_55_03.png: 416x640 2 defects, 17.4ms
image 37/40 /kaggle/working/data_split/test/images/f08e76ae-Shortfill_Image_14-01-2024_13_22_56.png: 416x640 2 defects, 17.4ms
image 38/40 /kaggle/working/data_split/test/images/f10d734d-Shortfill_Image_14-01-2024_13_50_53.png: 416x640 1 defect, 17.6ms
image 39/40 /kaggle/working/data_split/test/images/f56da050-Shortfill_Image_14-01-2024_13_40_14.png: 416x640 1 defect, 17.5ms
image 40/40 /kaggle/working/data_split/test/images/f7d2960e-Shortfill_Image_14-01-2024_13_07_06.png: 416x640 1 defect, 17.6ms
Speed: 2.4ms preprocess, 19.0ms inference, 10.9ms postprocess per image at shape (1, 3, 416, 640)
```

CHAPTER 8
RECOMMENDATIONS

Adaptable Work Schedule: Offer interns the flexibility to choose their working hours, accommodating their individual preferences and schedules. This empowers them to maintain a healthy work-life balance and optimize their productivity, which is particularly important when working on complex projects like Defect Pipe using Yolov9 detection systems.

Structured Learning Resources: Develop comprehensive learning materials to assist interns during their onboarding process and guide them through their skill development journey. These resources should cover essential topics related to the company's technologies, processes, and organizational culture. For example, interns can benefit from tutorials on DeepStream pipeline architecture and YOLOv9 implementation.

Guidance Program: Implement a mentorship program where each intern is paired with an experienced mentor who can offer guidance, feedback, and support throughout the internship. Mentors play a vital role in facilitating learning, nurturing professional growth, and helping interns navigate challenges effectively. Regular check-ins with mentors can provide ongoing encouragement, especially during complex tasks like implementing YOLOv9 for defect pipe detection.

Real-world Project Assignments: Assign interns to real-world projects that allow them to apply their knowledge and skills in practical scenarios. These projects should be challenging yet attainable, providing interns with opportunities to learn new concepts, solve complex problems, and showcase their abilities. For instance, interns could work on developing a defect pipe detection pipeline using YOLOv9, integrating it into existing systems for real-time monitoring.

Constructive Feedback and Evaluation: Offer interns regular feedback and evaluation sessions to assess their progress, identify areas for improvement, and acknowledge their accomplishments. Feedback sessions should be constructive, specific, and actionable, focusing on interns' strengths and areas for development. Encouraging interns to reflect on their experiences, set learning goals, and take ownership of their professional development can empower them to excel in their roles. Feedback discussions could delve into their proficiency with YOLOv9 and their contributions to the defect pipe detection project.

CHAPTER 9
CONCLUSION

- The Defect Pipe Detection application utilizing YOLOv9 ensures robust detection of anomalies in industrial pipelines, enhancing safety management by automating detection processes and facilitating timely responses to defects.
- By leveraging advanced computer vision algorithms, it enhances efficiency and accuracy, minimizing the risk of pipeline failures and accidents.
- The real-time detection capabilities allow prompt action, reducing downtime and ensuring pipeline safety and integrity.
- Automation reduces manual workload, minimizing human error and oversight while ensuring consistent monitoring of pipeline integrity.
- The user-friendly interface facilitates ease of operation, enhancing user adoption and integration into existing workflows.
- Utilizing DeepStream pipeline and YOLOv9 model optimizes computational resources for fast, reliable, and accurate detection, even in complex environments.
- Overall, the application represents a significant advancement in safety management, enabling proactive maintenance and ensuring industrial infrastructure reliability.

CHAPTER 10
REFERENCES

1. Founder : Kamalakar Devaki
   https://www.linkedin.com/in/kamalakardevaki/
2. Co-Founder : Jesudas Fernandes
   https://www.linkedin.com/in/jesudasfernandes/
3. Co-Founder : Ravi Kumar Rayana
   https://www.linkedin.com/in/ravi-kumar-rayana-05432121/
4. Co-Founder : Radhika Kanigiri
   https://www.linkedin.com/in/radhika-kanigiri-5b5706213/
5. Quickstart Guide- Installing Deepstream
   https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_Quickstart.html
6. Getting Started with NVIDIA DeepStream
   https://medium.com/analytics-vidhya/getting-started-with-nvidia-deepstream-5-0-8a05d061c4ee
7. Computer Vision in production –Nvidia DeepStream
   https://medium.com/virtuslab/computer-vision-in-production-nvidia-deepstream-7cb0b51af444
8. YoloV9 Training
   https://docs.ultralytics.com/models/yolov9/
9. A. B. Amjoud and M. Amrouch, "Object Detection Using Deep Learning, CNNs and Vision Transformers: A Review," in IEEE Access, vol. 11, pp. 35479-35516, 2023, doi: 10.1109/ACCESS.2023.3266093.
10. YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information
11. YOLOv9:Object Detection Model Explained
    https://encord.com/blog/yolov9-sota-machine-learning-object-dection-model/